ROCKET for Univariate Time Series Classification: Experimental Evaluation Against State-of-the-Art Baselines

Alessandro, Anton, Lara, Iñigo
Faculty of Informatics / UPV
San Sebastián, Spain

***Abstract*—In this paper, we present ROCKET, a univariate time series classification algorithm based on random convolutional kernels, which combines speed and high accuracy. We evaluate ROCKET on a selection of datasets from the UCR repository and compare its performance against seven benchmark algorithms: 1NN-Euclidean, 1NN-DTW, Shapelet Transform, BOSS ensemble, Time Series Forest, InceptionTime and catch22. Our experimentation employs a robust validation framework and considers multiple evaluation metrics, including accuracy and computational efficiency. The results demonstrate that ROCKET consistently outperforms the benchmark algorithms in both accuracy and scalability, demonstrating its effectiveness for large-scale time series classification tasks.**

***Index Terms*—Time series classification, Random convolutional kernels**

I. Introduction

Time series classification is a fundamental task in many domains where large volumes of sequential data are generated and need to be processed efficiently and accurately. Examples include financial markets, healthcare and industrial monitoring. Traditional approaches often achieve competitive accuracy but suffer from high computational cost and limited scalability when applied to large or complex datasets [1], [2].

ROCKET (RandOm Convolutional KErnel Transform) is an algorithm designed to overcome these limitations. It applies a large number of randomly initialized convolutional kernels to transform time series data into feature representations, which are then fed into linear classifiers. This approach enables both fast processing and high classification accuracy, even on large-scale datasets.

In this paper, we evaluate ROCKET on a selection of UCR datasets [3] and compare its performance with six benchmark algorithms, analyzing both accuracy and computational efficiency.

II. ROCKET Method

ROCKET transforms time series by applying convolutional kernels, which are not learned through training, but instead are randomly initialized [4]. Each kernel has parameters including length, weights, dilation, and bias, all of which are assigned randomly. The randomness allows the algorithm to generate a diverse set of filters, capable of capturing a wide variety of temporal patterns.

Once the kernels are applied to a time series, each convolution produces a feature map, showing the activation of the kernel across the entire series. From each feature map, ROCKET extracts two key summary features: Maximum value and Proportion of Positive Values (PPV).

Using a large number of kernels, ROCKET transforms each time series into a high-dimensional feature vector, which captures a large variety of temporal patterns without requiring kernel training.

These feature vectors are then fed into a linear classifier, typically Ridge Regression or Logistic Regression, which performs the final classification.

A. Kernels

ROCKET relies on a large number of convolutional kernels to extract meaningful features from time series. Each kernel acts like a small filter that highlights certain patterns in the data. Although the kernels are randomly initialized and not learned, their diversity allows the algorithm to extract rich and varied features from the time series. This random initialization enables ROCKET to efficiently explore a wide variety of temporal patterns without the computational cost of training each kernel.

The main parameters of each kernel are:

- Length l: determines how many consecutive points the kernel considers, which allows kernels of different scales to detect both short and long-term patterns.
- Weights w: sampled from a standard normal distribution, giving each kernel a unique pattern of emphasis over the input series.
- Dilation d: controls the spacing between points in the kernel. A dilation of $d > 1$ allows the kernel to skip points in the series, allowing the model to capture patterns that occur over longer time intervals or at multiple scales.
- Bias b: random offset added to the convolution output, which increases variability of the resulting feature map.

Additionally, padding is applied when necessary to ensure that kernels cover the full series, preserving feature extraction across all time points. Each kernel produces a feature map along the series, which will later be summarized in the feature extraction stage.

B. Time Series Transformation

The core of the ROCKET algorithm lies in the transformation of the input time series via the application of the randomly generated convolutional kernels. Given a univariate time series $X = (x_1, x_2, \ldots, x_n)$ and a random kernel with weights $W = (w_1, w_2, \ldots, w_l)$, length l, dilation d, and bias b, the time series transformation is achieved through a 1D convolution operation.The kernel slides across the time series, computing the dot product between its weights and the corresponding subsequences of the input. Because of the dilation parameter d, the kernel does not necessarily interact with strictly contiguous data points; rather, it spans a receptive field of $(l-1)d+1$. For a specific position i in the time series, the convolution output (activation) is calculated as:

$$a_i = \left(\sum_{j=0}^{l-1} x_{i+(j \cdot d)} \cdot w_{j+1} \right) + b$$

This operation is repeated across the entire sequence, producing a resulting vector known as a feature map, M. This map represents the degree to which the specific temporal pattern defined by the random kernel is present at each point in the time series. By applying a large ensemble of these kernels (typically 10,000 in the default configuration), ROCKET projects the original time series into a highly diverse, transformed spatial representation.

C. Feature Extraction

Passing the time series through 10,000 kernels results in 10,000 distinct feature maps. To make this high-dimensional output usable for a standard classifier without causing a combinatorial explosion in memory and processing time, ROCKET applies a pooling step to extract fixed-length scalar summaries from each feature map.For every feature map M, ROCKET extracts exactly two aggregate features:Maximum Value (Max): This captures the peak activation of the kernel across the entire time series. It acts as an indicator of the single strongest match between the kernel's pattern and the input data.

$$Max(M) = \max_i (M_i)$$

Proportion of Positive Values (PPV): This represents the frequency with which a pattern appears in the time series. It calculates the fraction of the feature map where the activation is strictly greater than zero.

$$PPV(M) = \frac{1}{|M|} \sum_{i=1}^{|M|} [M_i > 0]$$

(where $[\cdot]$ represents the Iverson bracket, equating to 1 if the condition is true and 0 otherwise).By extracting two features per kernel, an ensemble of 10,000 kernels produces a final, flat feature vector of 20,000 dimensions for each time series. This extraction mechanism is critical, as it allows ROCKET to handle input series of varying lengths while still outputting a uniform feature vector size for the classifier.

D. Classifier

Once the feature extraction phase is complete, the time series classification problem is effectively reduced to a standard tabular machine learning task. The 20,000-dimensional feature vectors are used to train a linear classifier. ROCKET typically employs a Ridge Regression classifier (for smaller datasets) or Logistic Regression (for larger datasets). The choice of a linear model is highly intentional: linear classifiers scale exceptionally well to high-dimensional data and are extremely fast to train. Because the random convolutional kernels combined with the non-linear PPV feature extraction inherently map the data into a space where the classes are largely linearly separable, complex non-linear classifiers (like deep neural networks or complex tree ensembles) are unnecessary. This synergy between diverse, random feature generation and a fast linear backend is the primary driver of ROCKET's computational efficiency.

E. Advantages

The ROCKET algorithm introduces several distinct advantages over traditional time series classification methods:

- Exceptional Computational Efficiency: By eliminating the need to update kernel weights via gradient descent, ROCKET dramatically reduces training time. It achieves state-of-the-art accuracy in a fraction of the time required by deep learning models like InceptionTime or ensemble methods like HIVE-COTE.
- High Accuracy: Despite its reliance on random initialization, the sheer volume and diversity of the kernels allow ROCKET to capture a highly discriminative feature set, consistently matching or outperforming computationally heavier benchmarks.
- Scalability: The linear nature of the classification phase and the independence of the kernel convolutions make ROCKET highly scalable to both long time series and datasets with massive sample sizes.
- Robustness to Hyperparameters: ROCKET requires almost no hyperparameter tuning. The default setting of 10,000 kernels is universally effective across diverse domains, removing the need for costly cross-validation searches.

F. Limitations

While highly effective, the original ROCKET algorithm does have specific constraints:

- Memory Consumption: Although computationally fast, storing the 20,000-dimensional feature vectors for every sample in a large dataset can lead to substantial RAM usage, which can be a bottleneck on memory-constrained hardware.
- Lack of Interpretability: Due to the random nature of the convolutional kernels, ROCKET acts largely as a "black box." It is exceedingly difficult to trace back exactly which temporal patterns or physical phenomena in the raw data drove the model's final classification decision.

- Non-Determinism: The random sampling of weights, biases, and dilations means that feature spaces will vary slightly between runs unless a strict random seed is applied, which can complicate exact reproducibility in some deployment environments.
- Univariate Constraint: The base ROCKET algorithm is designed explicitly for univariate time series, requiring architectural modifications (such as those seen in MultiROCKET) to effectively process multivariate data without losing cross-channel context.

III. Variants

Since its introduction, ROCKET has inspired a family of successors that address its main limitations—computational overhead, non-determinism, and restriction to univariate input—while preserving or improving upon its accuracy. The three principal variants are described below.

A. MiniROCKET

MiniROCKET (Dempster et al., 2021), optimizes the original framework by over an order of magnitude in computational speed without compromising classification accuracy. This variant transitions the model toward a nearly deterministic state by eliminating several stochastic components. In MiniROCKET, the kernel length is standardized to a fixed value of 9, and the weight space is constrained to only two discrete values. Unlike the original random initialization, the bias values are systematically derived from the distribution of the convolution outputs on the training data. Furthermore, the feature set is streamlined by discarding the "max" pooling operation, extracting only the Proportion of Positive Values (PPV). These structural constraints enable hardware-level optimizations that exploit the fixed parameters, providing a substantial reduction in training and inference latency while maintaining state-of-the-art performance across the UCR archive.

B. MultiROCKET

MultiROCKET (Tan et al., 2022), extends the framework to multivariate time series and introduces a significantly richer feature vocabulary. While maintaining the efficient weight scheme of MiniROCKET (utilizing the fixed pool of 84 binary vectors), MultiROCKET augments the feature extraction process by introducing three additional pooling operators beyond the original PPV and Max:

- **Mean of Positive Values (MPV)**: Captures the average activation strength, quantifying the intensity of a pattern match.
- **Mean of Indices of Positive Values (MIPV)**: Encodes temporal localization by identifying where in the series a pattern typically occurs.
- **Longest Stretch of Positive Values (LSPV)**: Measures the temporal persistence of a pattern through its longest contiguous activation.

These five operators produce $5 \times k$ features per channel, providing the linear classifier with critical information regarding the frequency, strength, location, and duration of detected patterns. For multivariate data with C channels, the transformation is applied independently to each channel and concatenated. This approach achieves a statistically significant improvement in mean rank across the UCR archive while preserving a linear computational complexity of $O(k \cdot n \cdot l \cdot C)$. By avoiding cross-channel combinatorial kernels, MultiROCKET remains highly scalable while capturing a more comprehensive representation of complex temporal dynamics.

C. Hydra

Hydra (Dempster et al., 2023) represents a conceptually distinct evolution of the ROCKET paradigm, combining ideas from random convolutional kernels with dictionary-based methods.

Rather than summarising each feature map with scalar pooling statistics, Hydra organises kernels into groups of g kernels sharing the same dilation and padding, and builds a histogram of nearest-kernel assignments across each group and each position in the series. This histogram — counting, for each kernel within a group, how many time-series positions are most similar to that kernel's weights — serves as the feature representation passed to the linear classifier.

This construction is closely related to the Bag-of-SFA-Symbols (BOSS) dictionary representation: the group of g kernels can be seen as a soft codebook, and the histogram as a symbolic frequency profile. The key difference is that Hydra's codebook is random and extremely cheap to compute, whereas BOSS requires a Symbolic Fourier Approximation discretisation step that is quadratic in training-set size. Empirically, Hydra surpasses ROCKET and MultiROCKET in mean classification rank on the UCR archive while remaining an order of magnitude faster than InceptionTime. It is particularly effective on problems with long and complex series, where the positional frequency of matched patterns — rather than just their peak value or overall proportion — carries the most discriminative information.

A combined variant, **Hydra-MultiROCKET**, concatenates Hydra's histogram features with MultiROCKET's pooling features, achieving the highest mean accuracy in the ROCKET family to date on the UCR archive.

IV. Experimentation

A. Datasets

1) We try to choose datasets from a range of different domains. In total, we include datasets from 9 different domains in our experimentation. The selection includes device/energy monitoring (Computers), biomedical signals (ECG200, EMOPain), industrial sensors (FordA, Earthquakes, Trace), spectroscopy (EthanolLevel, Beef), biology/motion capture (Worms, WormsTwoClass), image (SwedishLeaf, MiddlePhalanxOutlineCorrect), traffic (Chinatown), and synthetic benchmarks (CBF, SyntheticControl).
2) We also take into account the length of the time series contained in the datasets. The lengths range from 24 (Chinatown) to 1751 (EthanolLevel).

3) Dataset size: Since we do not use the predefined train/test split, we are interested in the total number of samples in the datasets. These span over a range of 60 (Beef) to 5000 (TwoPatterns).
4) We also aim to reach a broad range of classes: Our selection mainly covers 2-class datasets, but we also include datasets with 3-6 classes (Worms, EMOPain, EthanolLevel, CBF, SyntheticControl, Beef, Trace). We also include a 15 class dataset (SwedishLeaf).
5) We also try to include datasets with different difficulties. SwedishLeaf has 15 classes which are perfectly balanced, which makes the dataset overall more challenging for classification. On the contrary, we also include unbalanced datasets like Earthquakes.

B. Evaluated Algorithms

To evaluate our algorithm, ROCKET and its variations MiniROCKET, MultiROCKET and MultiROCKETHydra, we run experiments with seven benchmark classifiers. We include simple distance based classifiers, which classify by Nearest Neighbour, using Euclidian Distance (1NN-ED) and Dynamic Time Warping (1NN-DTW). From the shapelet based algorithms, we select Shapelet Transform (ST). We also include a dictionary based classifier with BOSS. We also choose Time Series Forest (TSF), which takes an interval based approach and a feature based classifier with catch22. Lastly, we include InceptionTime, a deep learning model. All classifiers are loaded from the aeon toolkit and run with default parameters. The only exception is InceptionTime. For computational feasibility, we adjust the number of training epochs to 100 (default is 1500).

C. Validation Framework and Metrics

To ensure results that are robust, we evaluate all named algorithms with stratified 5-Fold Cross Validation. After loading the datasets from the UCR archive via aeon, we merge the train and test set. After, we repartition the samples and ensure that the classes are in equal proportion across the splits. For reproducibility, we fix the random seed to 42.

The Performance was measured by using four metrics: accuracy, balanced accuracy, weighted F1-score and Cohen's kappa. For datasets with a significant class imbalance, we are particularly interested in F1 score and Cohen's kappa.

Additionally, we measure training and inference time.

D. Hardware

To run the experiments, we distribute dataset-level jobs across multiple machines and execute all 5 folds for each classifier. Most algorithms are executed on CPU. The only exception is InceptionTime, which is trained on GPU using NVIDIA GeForce RTX 2080 hardware.

V. Results and Discussion

VI. Conclusions

References

[1] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The great time series classification bake off: A review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, vol. 31, pp. 606–660, 2017.

[2] M. Middlehurst, P. Sch"afer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, pp. 1958–2031, 2024.

[3] U. T. S. C. Repository, "Time series classification repository," https://www.timeseriesclassification.com/, 2026, accessed: 2026-03-28.

[4] A. Dempster, F. Petitjean, and G. I. Webb, "Rocket: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1454–1495, 2020.